

Київський національний університет
імені Тараса Шевченка

Факультет комп'ютерних наук та кібернетики

Звіт

до лабораторної роботи №1
на тему:

**«Розробка клієнта для взаємодії з віддаленою папкою з файлами. Етап 4,
варіант 8-6»**

Студента четвертого курсу
групи МІ-41
Мекшакова Артемія Олексійовича

Київ-2026

Зміст

1	Постановка задачі	1
1.1	Умови вимірювань	1
2	Спільна основа	2
3	Порівняння за критеріями	3
3.1	Доступ до файлів і відстеження змін	5
3.2	Доставка й оновлення	5
3.3	Безпека	5
3.4	Розмір, запуск і синхронізація	6
4	Однакові сценарії у двох клієнтах	6
5	Переваги й обмеження	8
5.1	Десктопний клієнт	8
5.2	Веб-клієнт	9
5.3	Коли що обрати	9
6	Висновки	9

1 Постановка задачі

Четвертий етап лабораторної роботи присвячено порівнянню двох клієнтів системи Shelf, створених на попередніх етапах: десктопного застосунку на Electron (етап 2) і веб-клієнта на Next.js (етап 3). Обидва клієнти реалізують ту саму UML-модель етапу 1 і працюють з одним сервером через його REST API, тож користувач бачить у них той самий простір із файлами, ті самі операції варіанта 8-6 (сортування за назвою, фільтр «усі файли / лише .cpp / лише .png», перегляд .kt як тексту і .jpg як зображення) і ту саму синхронізацію з локальною папкою.

Клієнти порівнюються за такими критеріями: технологія і середовище виконання; як користувач отримує застосунок і його оновлення; підтримувані платформи; доступ до локальних файлів і відстеження змін у папці; де зберігається стан синхронізації; модель безпеки; розмір артефакту, час запуску і тривалість першої синхронізації (ці три показники виміряно); тестування.

Адреси проекту:

- сайт: <https://shelf-opal-two.vercel.app>;
- REST API: <https://shelf-api-1989.onrender.com>, документація за шляхом /api/docs;
- вихідний код: <https://github.com/NONAME-lf/shelf>.

1.1 Умови вимірювань

Усі числа в звіті отримано 30.09.2026 на одному комп'ютері: Apple M2 Max, 32 ГБ оперативної пам'яті, macOS 26.6.2, Node.js 24.11.1, Playwright 1.63 з Chromium 153 і Electron 39.8.10. Кожен час у звіті є медіаною трьох запусків. Розміри подано так само, як у таблиці файлів самого клієнта: 1 КБ = 1024 Б, 1 МБ = 1024 КБ. На іншому комп'ютері чи з іншим підключенням

до інтернету значення будуть іншими, тому варто дивитися передусім на співвідношення між клієнтами.

Повторити вимірювання можна скриптом `tools/compare-clients.mjs`, який друкує значення кожного запуску і медіану. Командам `artefacts`, `startup` і `sync` потрібні зібрані клієнти, а `sync` ще й запущений локальний стек:

```
pnpm -F @shelf/shared build && pnpm -F @shelf/desktop dist:mac
pnpm -F @shelf/web build # веб-клієнт для локального API
node tools/compare-clients.mjs loc # і artefacts, web-load, startup
node tools/compare-clients.mjs sync # потрібен запущений pnpm stack:up
```

Показник	Спосіб вимірювання	Команда
частка спільного коду	рядки файлів <code>.ts</code> і <code>.tsx</code> зі списку <code>git ls-files</code> у каталогах <code>src</code> , пораховані так само, як це робить утиліта <code>wc</code> ; тести й тестові допоміжні файли окремо	<code>loc</code>
розмір десктопного артефакту	розмір <code>.dmg</code> і сума розмірів файлів <code>Shelf.app</code> , щойно зібраних з поточного коду	<code>artefacts</code>
обсяг першого завантаження сайту	Chromium з порожнім профілем і вимкненим кешем; сума <code>encodedDataLength</code> усіх відповідей за протоколом DevTools, тобто передані байти разом із заголовками	<code>web-load</code>
запуск десктопа	від старту зібраного <code>Shelf.app</code> через Playwright <code>_electron</code> (помічник <code>apps/desktop/scripts/lib/app.mjs</code>) до видимої кнопки входу; щоразу новий профіль	<code>startup</code>
запуск веба	від переходу на <code>/login</code> опублікованого сайту в уже запущеному Chromium з порожнім профілем до видимої кнопки входу; сервер попередньо розбуджено запитами <code>/api/health</code>	<code>startup</code>
перша синхронізація	локальний стек; новий обліковий запис із 10 демонстраційними файлами (597 КБ); від натискання «Синхронізувати» до рядка «Скачано з сервера: 10»; щоразу порожня папка: тимчасовий каталог у десктопі, каталог OPFS у вебі	<code>sync</code>

2 Спільна основа

Проект є монорепозіторієм `pnpm`: сервер `apps/api`, клієнти `apps/desktop` і `apps/web` та два спільні пакети. Пакет `@shelf/shared` не залежить ні від `React`, ні від `Electron` чи браузера. У ньому типи й DTO (їх використовує і сервер), операції варіанта `sortByName`, `filterByType` і `getVisibleFiles`, `toggleColumn`, вибір способу перегляду `previewKindOf` і `createPreview`, REST-клієнт `FileApiClient`, перевірка ліміту `splitBySize` і вся логіка синхронізації: `computeStatus`, побудова плану і `SyncEngine` з інтерфейсами `LocalFolder` і `SnapshotStore`. Пакет `@shelf/ui` містить `React`-компоненти всіх екранів, від `AuthForm` і `WorkspaceLayout` до `FileTableView`, `PreviewDialog`, `SyncPanel` і `ConflictDialog`, а також хук `useFileListController`, яким реалізовано керівний клас `FileListController` з VOPC-діаграми. Клієнт лише складає з цих частин свої екрани і реалізує інтерфейси для своєї платформи.

Пакет	Файлів коду	Рядків коду	Файлів тестів	Рядків тестів
@shelf/shared	15	866	13	1127
@shelf/ui	18	1152	1	17
@shelf/desktop	19	1184	8	623
@shelf/web	22	1341	9	1129
@shelf/api (для довідки)	29	839	8	599

Разом зі спільними пакетами десктопний клієнт має 3202 рядки коду: 2018 спільних (63,0 %) і 1184 власних (37,0 %). У веб-клієнті 3359 рядків, з них ті самі 2018 спільних (60,1 %) і 1341 власний (39,9 %). Більшість власного коду обох клієнтів припадає на роботу з локальною папкою. У десктопі це main-процес, 640 рядків: NodeLocalFolder, FolderWatcher, SyncService, SettingsStore, збереження файлів і обробники IPC. У вебі це каталог sync/, 855 рядків: BrowserLocalFolder, сховища в IndexedDB і WebSyncService з опитуванням папки, запитами дозволу і перекладом помилок браузера. Веб потребує тут більше коду, бо мусить обходити обмеження браузера. До тестових файлів зараховано й допоміжні: фіктивний API і файлову систему в пам'яті.

Операції варіанта поводяться в обох клієнтах однаково не через старання дублювання, а тому, що це одна й та сама функція. Екран кожного клієнта викликає useFileListController з @shelf/ui, а той обчислює видимі рядки як getVisibleFiles(files, direction, filter), тобто filterByType(sortByName(files, direction), filter) з @shelf/shared; клієнт лише передає список файлів, отриманий від FileApiClient. Так само спільні toggleColumn і previewKindOf. Тести цих функцій (17 для розширень, сортування й фільтра, 15 для перегляду, 3 для стовпців) лежать у @shelf/shared і перевіряють саме той код, який потрапляє в обидві збірки, а наскрізні перевірки обох клієнтів однаково вмикають спадання з «Лише .cpp» і очікують matrix.cpp перед geometry.cpp. Від середовища залежить тільки мова порівняння в Intl.Collator: її задає Electron або браузер, тому порядок кириличних і латинських назв визначає мова системи.

3 Порівняння за критеріями

Таблиця зводить відмінності клієнтів; виміряні значення отримано способами, описаними в підрозділі 1.1.

Критерій	Десктоп (етап 2)	Веб (етап 3)
технологія	Electron 39 (Chromium і Node.js), React 19, збірка electron-vite; процеси main, preload і render	Next.js 16 з App Router, React 19; усі сторінки статичні, код виконує браузер
як отримати	непідписаний .dmg; перший запуск через пункт «Відкрити» контекстного меню	адреса сайту, встановлення не потрібне
оновлення	нова збірка dist:mac і повторне встановлення; перевірки нових версій немає	Vercel сам збирає і публікує сайт після push у репозиторій; нову версію користувач отримує під час наступного відкриття

Критерій	Десктоп (етап 2)	Веб (етап 3)
платформи	зібрано для macOS (arm64); Windows (dist:win, NSIS) і Linux (AppImage) описано в <code>electron-builder.yml</code>	будь-який сучасний браузер, зокрема на телефоні; синхронізація з папкою лише в Chrome і Edge
екран	окреме вікно, щонайменше 1024 × 640	вкладка браузера; вужче 768 px бічна панель ховається за кнопкою меню
доступ до файлів	Node fs у main-процесі; будь-яка папка із системного діалогу, відомий повний шлях	File System Access API: handle папки без шляху, дозвіл користувача; після перезавантаження сторінки дозвіл треба підтвердити знову
відстеження змін	події файлової системи (chokidar), запуск через 1,5 с після останньої зміни	опитування списку файлів кожні 5 с, лише поки вкладка видима
час зміни скачаного файлу	utimes задає час серверної версії	задати неможливо: файл має поточний час, і саме його записано у знімок
знімок і налаштування	JSON-файли в <code>userData: settings.json</code> і <code>snapshots/<sha1 шлях>.json</code>	IndexedDB: IndexedDbSnapshotStore і FolderBindingStore
скачування	діалог «Зберегти як» і перетягування рядка у Finder	Blob і <code><a download></code> , файл потрапляє до теки завантажень браузера
перетягування	в обидва боки: файли у вікно і рядок із вікна	лише файли на сторінку
безпека	<code>contextIsolation</code> , <code>sandbox</code> , типізований міст <code>window.shelf</code> у <code>preload</code> , CSP, заборона навігації вікна	пісочниця браузера, доступ лише до обраної папки, CORS на сервері
адреса API	поле «Адреса сервера» на екрані входу, зберігається в <code>settings.json</code>	змінна <code>NEXT_PUBLIC_API_URL</code> , вбудована під час збірки
запуск без мережі	вікно відкривається, збережена сесія не скидається, але список файлів і синхронізація недоступні	опублікована сторінка не відкривається (service worker немає)
розмір артефакту	<code>.dmg</code> 104,8 МБ; <code>Shelf.app</code> 255,3 МБ, з них Electron Framework 252,6 МБ і власний <code>app.asar</code> 1,1 МБ	перше відкриття <code>/login</code> 163,4 КБ (11 запитів), <code>/workspace</code> 171,4 КБ (15 запитів)
запуск до форми входу	469 мс; перший запуск після збірки 2,3 с	363 мс з порожнім кешем, 110 мс з кешем браузера
перша синхронізація 10 файлів	216 мс	202 мс (папка OPFS)
unit-тести	Vitest, 7 файлів, 42 тести: <code>NodeLocalFolder</code> , <code>FolderWatcher</code> , <code>SyncService</code>	Vitest, 7 файлів, 56 тестів: <code>BrowserLocalFolder</code> , <code>IndexedDB</code> , <code>WebSyncService</code>
наскрізні перевірки	Playwright <code>_electron: smoke.mjs</code> на 12 кроків і <code>check-packaged.mjs</code> для зібраного <code>Shelf.app</code>	Playwright з Chromium, 20 кроків, папка в OPFS; пройдено і на опублікованій системі

Спільні 142 тести `@shelf/shared` і 2 тести `@shelf/ui` перевіряють код обох клієнтів одночасно; разом із тестами клієнтів і 58 тестами сервера `pnpm test` виконує 300 тестів, і всі вони проходять.

3.1 Доступ до файлів і відстеження змін

Найбільша відмінність клієнтів полягає в доступі до локальної папки. Main-процес десктопа має Node fs: він працює з будь-якою папкою, яку користувач обрав у системному діалозі, знає її повний шлях і записує скачаний файл через тимчасовий `.shelf-*.part`, `utimes` і перейменування. Файл отримує час зміни серверної версії, тож наступне порівняння одразу бачить його незміненим. `FolderWatcher` отримує від системи події про зміни, чекає, доки запис завершиться, і запускає синхронізацію через 1,5 с після останньої зміни, навіть коли вікно згорнуте.

Сторінка в браузері отримує лише `handle` обраної папки без шляху, а доступ до неї дає користувач. Після перезавантаження Chrome пам'ятає папку, але дозвіл треба підтвердити знову, і попросити його сторінка може тільки у відповідь на клік, тому автоматичні запуски чекають. Змінити час файлу браузер не дозволяє, отже `SyncEngine` записує у знімок час, який повернув `LocalFolder.write`, інакше скачаний файл виглядав би зміненим локально. Подій про зміни в папці браузер не надсилає, тож `WebSyncService` перечитує її вміст кожні 5 с: зміна потрапляє на сервер із затримкою до 5 с, а в прихованій чи закритій вкладці не потрапляє зовсім.

3.2 Доставка й оновлення

Щоб отримати десктоп, користувач скачує `.dmg` на 104,8 МБ і переносить `Shelf` до теки «Програми». Збірку не підписано сертифікатом Apple, тому вперше її відкривають через контекстне меню. Оновити застосунок можна тільки новою збіркою, бо перевірки версій у ньому немає. Натомість встановлена програма не залежить від хостингу сайту, а сервер для неї можна змінити просто на екрані входу: локальний стек чи опублікований API.

Для веба нічого встановлювати не треба: перше відкриття сторінки входу передає 163,4 КБ. Після `push` у репозиторій `Vercel` сам збирає й публікує нову версію, і всі користувачі отримують її під час наступного відкриття сторінки. Ціна цього полягає в залежності від `Vercel` і `Render`: безкоштовний сервер `Render` засинає після 15 хвилин без запитів, а адресу API вбудовано в збірку, тож інший сервер потребує іншої збірки.

3.3 Безпека

У десктопі інтерфейс працює в `renderer`-процесі з `contextIsolation` і `sandbox`, без доступу до Node. До файлів і системних діалогів він звертається лише через 13 методів моста `window.shelf`, описаних типом `ShelfBridge`, а назви файлів перевіряє `isSafeFileName`. CSP у `index.html` дозволяє скрипти тільки із самого застосунку, вікно не може перейти на іншу адресу, а зовнішні посилання відкриваються в системному браузері. Проте сам застосунок має всі права користувача, і непідписаній збірці доводиться просто довіряти.

Сторінку веб-клієнта обмежує пісочниця браузера: вона бачить лише обрану користувачем папку, а дозвіл прив'язано до походження сайту. API розташовано на іншому домені, і відповіді сторінка читає з дозволу CORS; сервер приймає будь-яке походження, бо сесію передає токен у заголовок `Authorization`, а не `cookie`. В обох клієнтах JWT лежить у `localStorage`, тому

ризик його викрадення через вразливість XSS однаковий; зменшують його екранування React і відсутність сторонніх скриптів.

3.4 Розмір, запуск і синхронізація

Образ .dmg приблизно в 657 разів більший за перше завантаження сторінки входу, але власного коду в ньому небагато: зі 255,3 МБ Shelf.app 252,6 МБ займає Electron Framework (Chromium і Node.js), а app.asar лише 1,1 МБ. Веб-клієнт використовує вже встановлений браузер: для /login сайт передає 155,4 КБ скриптів (494,0 КБ після розпакування), а перехід після входу на /workspace додає ще 11,5 КБ разом із відповідями API. Таблиці First Load JS next build у Next.js 16 уже не друкує, тому розмір веб-клієнта оцінено саме переданими байтами.

Від старту зібраного застосунку до форми входу минає 469 мс (запуски 2304, 469 і 420 мс). Перший запуск одразу після збірки триває 2,3 с, імовірно, через перевірку нового виконуваного файлу системою; повторна серія дала 431, 527 і 436 мс. Сторінка входу опублікованого сайту з порожнім кешем з'являється за 363 мс (377, 363 і 352 мс), а з кешем за 110 мс; повторна серія дала 398 і 105 мс. Час веба не містить запуску браузера, а час десктопа містить старт процесів Electron, тому для користувача з уже відкритим браузером сайт відкривається не повільніше за застосунок.

Перша синхронізація 10 демонстраційних файлів із локального стека в порожню папку триває 216 мс у десктопі (224, 215 і 216 мс) і 202 мс у веб-і (200, 202 і 202 мс). Різниця невелика, бо час визначають запити до сервера, а не запис кількох сотень кілобайт. Веб вимірювано з каталогом OPFS: системний діалог вибору папки Playwright відкрити не може. У справжній папці Chrome записує файл через тимчасовий .cstswap і може перевіряти його перед заміною, тому там час може бути більшим; окремо це не вимірювалося.

4 Однакові сценарії у двох клієнтах

Нижче поруч розміщено фрагменти знімків того самого екрана з етапів 2 і 3: ліворуч десктоп, праворуч веб. Знімки зроблено у вікні шириною 1280 px на локальному стеку зі свіжими демонстраційними даними, тому дати в таблицях різні. Рис. 1 і 3 показують ліву верхню частину вікна з бічною панеллю і першими стовпцями таблиці, рис. 2 показує сам діалог.

Десктоп (Electron)

Shelf					
Простір: Артем artem@shelf.dev					
СИНХРОНІЗАЦІЯ					
Локальна папка: /tmp/Shelf-HqStqB					
Змінити Синхронізувати					
Відстежувати зміни автоматично					
Стовпці таблиці					
☒ Тип ☒ Розмір ☒ Дата створення ☒ Дата зміни ☒ Хто завантажив ☒ Хто редагував					
НАЗВА	ТИП	РОЗМІР	ДАТА СТОВРЕННЯ	ДАТА ЗМІНИ	
backup.zip	.zip	576 Б	29.09.2026 22:32	29.09.2026 22:32	
class-diagram.png	.png	564 КБ	29.09.2026 22:32	29.09.2026 22:32	
geometry.cpp	.cpp	226 Б	29.09.2026 22:32	29.09.2026 22:32	
Main.kt	.kt	319 Б	29.09.2026 22:32	29.09.2026 22:32	
matrix.cpp	.cpp	448 Б	29.09.2026 22:32	29.09.2026 22:32	
mountains.jpg	.jpg	30 КБ	29.09.2026 22:32	29.09.2026 22:32	
README.md	.md	330 Б	29.09.2026 22:32	29.09.2026 22:32	
shelf-logo.png	.png	1,3 КБ	29.09.2026 22:32	29.09.2026 22:32	
todo.txt	.txt	185 Б	29.09.2026 22:32	29.09.2026 22:32	
3air.txt	.txt	148 Б	29.09.2026 22:32	29.09.2026 22:32	

Веб (Chromium)

Shelf					
Простір: Артем artem@shelf.dev					
СИНХРОНІЗАЦІЯ					
Локальна папка: Не прив'язано					
Обрати папку					
Синхронізувати					
Відстежувати зміни автоматично					
Стовпці таблиці					
☒ Тип ☒ Розмір ☒ Дата створення ☒ Дата зміни ☒ Хто завантажив ☒ Хто редагував					
НАЗВА	ТИП	РОЗМІР	ДАТА СТОВРЕННЯ	ДАТА ЗМІНИ	
backup.zip	.zip	576 Б	30.09.2026 14:43	30.09.2026 14:43	
class-diagram.png	.png	564 КБ	30.09.2026 14:43	30.09.2026 14:43	
geometry.cpp	.cpp	226 Б	30.09.2026 14:43	30.09.2026 14:43	
Main.kt	.kt	319 Б	30.09.2026 14:43	30.09.2026 14:43	
matrix.cpp	.cpp	448 Б	30.09.2026 14:43	30.09.2026 14:43	
mountains.jpg	.jpg	30 КБ	30.09.2026 14:43	30.09.2026 14:43	
README.md	.md	330 Б	30.09.2026 14:43	30.09.2026 14:43	
shelf-logo.png	.png	1,3 КБ	30.09.2026 14:43	30.09.2026 14:43	
todo.txt	.txt	185 Б	30.09.2026 14:43	30.09.2026 14:43	
3air.txt	.txt	148 Б	30.09.2026 14:43	30.09.2026 14:43	

Рисунок 1. Простір користувача: ліворуч десктоп, праворуч веб

На рис. 1 збігаються компоновка, бічна панель, фільтр, стовпці і значення в рядках: обидва екрани складено з тих самих компонентів WorkspaceLayout, TypeFilterControl, FileTableView і ColumnPicker. Різняться тільки блок синхронізації. Десктопу папку прив'язано заздалегідь, і він показує її повний шлях /tmp/Shelf-HqStqB та кнопку «Змінити». У веб-версії папку ще не обрано, тож видно «Не прив'язано» і кнопку «Обрати папку»; після вибору там з'явиться лише назва папки, бо шляху сторінка не знає.

Десктоп (Electron)

Конфлікти версій

Ці файли змінилися і в локальній папці, і на сервері після останньої синхронізації. Оберіть, яку версію залишити; новішу позначено за замовчуванням.

Файл	ЛОКАЛЬНА ВЕРСІЯ	ВЕРСІЯ НА СЕРВЕРІ
todo.txt	☐ Залишити локальну 29.09.2026 22:27, 80 Б	☒ Залишити з сервера 29.09.2026 22:32, 66 Б

Скасувати синхронізацію Застосувати

Веб (Chromium)

Конфлікти версій

Ці файли змінилися і в локальній папці, і на сервері після останньої синхронізації. Оберіть, яку версію залишити; новішу позначено за замовчуванням.

Файл	ЛОКАЛЬНА ВЕРСІЯ	ВЕРСІЯ НА СЕРВЕРІ
todo.txt	☒ Залишити локальну 30.09.2026 14:44, 114 Б	☐ Залишити з сервера 30.09.2026 14:44, 66 Б

Скасувати синхронізацію Застосувати

Рисунок 2. Діалог конфліктів версій: ліворуч десктоп, праворуч веб

Діалог ConflictDialog на рис. 2 в обох клієнтах однаковий: заголовок, пояснення, стовпці «Локальна версія» і «Версія на сервері» з часом і розміром кожної, кнопки «Скасувати синхронізацію» і «Застосувати». Заздалегідь вибрано новішу версію за тим самим правилом із @shelf/shared. У сценарії етапу 2 новішою була серверна версія todo.txt (22:32, 66 Б проти локальної 22:27, 80 Б), а в сценарії етапу 3 локальний файл змінили вже після серверного (обидва о 14:44; 114 Б проти 66 Б), тому позначки стояли у різних стовпцях.

Десктоп (Electron)

НАЗВА	ТИП	РОЗМІР	ДАТА СТВОРЕННЯ	ДАТА ЗМІНИ
backup.zip	.zip	576 Б	29.09.2026 22:32	29.09.2026 22:32
class-diagram.png	.png	564 КБ	29.09.2026 22:32	29.09.2026 22:32
geometry.cpp	.cpp	226 Б	29.09.2026 22:32	29.09.2026 22:32
Greeting.kt	.kt	49 Б	29.09.2026 22:32	29.09.2026 22:32
lab-notes.txt	.txt	95 Б	29.09.2026 22:32	29.09.2026 22:32
Main.kt	.kt	319 Б	29.09.2026 22:32	29.09.2026 22:32
matrix.cpp	.cpp	448 Б	29.09.2026 22:32	29.09.2026 22:32
mountains.jpg	.jpg	30 КБ	29.09.2026 22:32	29.09.2026 22:32
README.md	.md	330 Б	29.09.2026 22:32	29.09.2026 22:32
shelf-logo.png	.png	1,3 КБ	29.09.2026 22:32	29.09.2026 22:32
todo.txt	.txt	66 Б	29.09.2026 22:32	29.09.2026 22:32
3air.txt	.txt	148 Б	29.09.2026 22:32	29.09.2026 22:32

Веб (Chromium)

НАЗВА	ТИП	РОЗМІР	ДАТА СТВОРЕННЯ	ДАТА ЗМІНИ
backup.zip	.zip	576 Б	30.09.2026 14:43	30.09.2026 14:43
class-diagram.png	.png	564 КБ	30.09.2026 14:43	30.09.2026 14:43
geometry.cpp	.cpp	226 Б	30.09.2026 14:43	30.09.2026 14:43
Greeting.kt	.kt	49 Б	30.09.2026 14:44	30.09.2026 14:44
lab-notes.txt	.txt	95 Б	30.09.2026 14:44	30.09.2026 14:44
Main.kt	.kt	319 Б	30.09.2026 14:43	30.09.2026 14:43
matrix.cpp	.cpp	448 Б	30.09.2026 14:43	30.09.2026 14:43
mountains.jpg	.jpg	30 КБ	30.09.2026 14:43	30.09.2026 14:43
README.md	.md	330 Б	30.09.2026 14:43	30.09.2026 14:43
shelf-logo.png	.png	1,3 КБ	30.09.2026 14:43	30.09.2026 14:43
todo.txt	.txt	114 Б	30.09.2026 14:43	30.09.2026 14:44
3air.txt	.txt	148 Б	30.09.2026 14:43	30.09.2026 14:43

Рисунок 3. Звіт після розв'язання конфлікту: ліворуч десктоп, праворуч веб

Рис. 3 показує результат «Застосувати». Звіт SyncReportView має ті самі чотири рядки, а «Без змін: 11» і «Конфліктів: 1» збігаються, бо план виконав той самий SyncEngine. Відрізняється напрямком: десктоп залишив версію з сервера («Скачано з сервера: 1», todo.txt має 66 Б), а веб локальну («Завантажено на сервер: 1», todo.txt має 114 Б і новий час зміни). Запис папки в бічній панелі знову різний: повний шлях у десктопі і лише назва Shelf у веб. Спільний рушій отримав від кожного клієнта тільки інші LocalFolder і SnapshotStore.

5 Переваги й обмеження

5.1 Десктопний клієнт

Переваги:

- працює з будь-якою папкою без повторних дозволів і реагує на зміни в ній подіями файлової системи, навіть коли вікно згорнуте;
- скачані файли мають час зміни серверної версії;
- файл можна зберегти діалогом «Зберегти як» або перетягнути рядок у Finder;
- адресу сервера можна змінити на екрані входу, а без мережі вікно все одно відкривається.

Обмеження:

- треба скачати 104,8 МБ і встановити непідписаний застосунок;
- кожне оновлення вимагає нової збірки і повторного встановлення;
- на диску займає 255,3 МБ, з яких власного коду лише 1,1 МБ;
- зібрано й перевірено тільки версію для macOS.

5.2 Веб-клієнт

Переваги:

- нічого не треба встановлювати: перше відкриття передає 163,4 КБ і показує форму входу за 363 мс;
- нова версія доступна всім одразу після публікації;
- працює на будь-якому пристрої з сучасним браузером, зокрема на телефоні;
- пісочниця браузера пускає сторінку лише до папки, яку дозволив користувач.

Обмеження:

- синхронізація з папкою працює лише в Chrome і Edge, а дозвіл після перезавантаження сторінки треба підтвердити знову;
- зміни в папці помічає опитування кожні 5 с і лише у видимій вкладці;
- скачаний файл отримує поточний час, а перетягнути рядок у файловий менеджер неможливо;
- без мережі сторінка не відкривається, а адресу API задає збірка.

5.3 Коли що обрати

Десктоп доречний, коли з локальною папкою працюють постійно й багато: вона може лежати будь-де, зміни мають потрапляти на сервер одразу, навіть із згорнутим вікном, а файли зручно перетягувати у Finder. Він же підходить для роботи з кількома серверами, наприклад з локальним стеком. Веб доречний, коли потрібен швидкий доступ до файлів без встановлення: з чужого комп'ютера, з телефона чи для того, щоб переглянути, скачати або завантажити кілька файлів. Для синхронізації у вебі потрібен Chrome або Edge і відкрита вкладка. Обидва клієнти працюють з тим самим простором, тому їх можна поєднувати: десктоп синхронізує робочу папку, а веб дає доступ до тих самих файлів з будь-якого пристрою.

6 Висновки

На етапі 4 порівняно десктопний і веб-клієнти Shelf. Обидва реалізують одну UML-модель і працюють з одним сервером, а спільні пакети становлять 63,0 % коду десктопа і 60,1 % коду веба, тому операції варіанта, перегляд, REST-клієнт, рушій синхронізації й усі екрани в них однакові. Відмінності зосереджено там, де платформа дає різні можливості. Десктоп має повний доступ до папки, події змін і збереження часу файлу, але поширюється як .dmg на 104,8 МБ і оновлюється перевстановленням. Веб не потребує встановлення: форма входу з'являється за 363 мс після передачі 163,4 КБ, а нові версії публікуються автоматично; проте папку він синхронізує лише в Chrome і Edge, опитуючи її кожні 5 с. Перша синхронізація 10 файлів триває майже однаково: 216 і 202 мс. Отже, десктоп краще підходить для постійної роботи з локальною папкою, а веб — для швидкого доступу до файлів з будь-якого пристрою.